SHOGUN-SYNC

Total War: Shogun 2 Multiplayer Synchronization Tool

Technical Specification

Architecture, Cryptography and Licensing Manual

Author: KW
Module: Architecture (`tech_spec.pdf`)

Contents

1 System Genesis & Architecture **2**
1.1 The Shogun 2 Desync Problem . . . 2
1.2 WYAG Inspiration (Write Yourself a Git) . . . 2

2 Cryptographic Subsystem: Custom SHA-1 **2**
2.1 Padding & Equations (from Design Notes) . . . 3
2.2 Message Expansion & Compression Rounds . . . 3
2.3 Compression Step Diagram . . . 3

3 Compression Subsystem: Zlib **3**
3.1 Functions . . . 4

4 Networking Subsystem: WebSocket & Auto-Resync **4**
4.1 Auto-Resync Frame Format . . . 4

5 Complete Function and Data Structure Specifications **4**
5.1 Data Structures . . . 4
5.2 Module Inventory . . . 5
5.2.1 `main.c` . . . 5
5.2.2 `kw_conf.c` . . . 5
5.2.3 `kw_conn.c` . . . 5
5.2.4 `kw_sha1.c` . . . 6

6 Third-Party Technologies & Licenses **6**
6.1 Zlib General Purpose Compression Library . . . 6
6.2 Mongoose Embedded Networking Library . . . 6

1 System Genesis & Architecture

1.1 The Shogun 2 Desync Problem

In multiplayer campaigns of *Total War: Shogun 2*, lockstep simulation issues often cause save files to diverge between peers. When this happens, the autosave file in:

`%APPDATA%/TheCreativeAssembly/Shogun2/save_games_multiplayer`

becomes out of sync, breaking the campaign. **Shogun-Sync** is a lightweight tool written in C designed to monitor save files, compare cryptographic hashes over WebSockets, and automatically transfer authoritative saves from the host to the client upon desynchronization.

1.2 WYAG Inspiration (Write Yourself a Git)

The internal architecture of Shogun-Sync is directly inspired by Thibault Polge's *Write Yourself a Git* (https://wyag.thb.lt/).

Instead of copying saves directly, Shogun-Sync implements a Git-like Content-Addressable Storage (CAS) database inside the directory:

`%APPDATA%/TheCreativeAssembly/Shogun2/.sync`

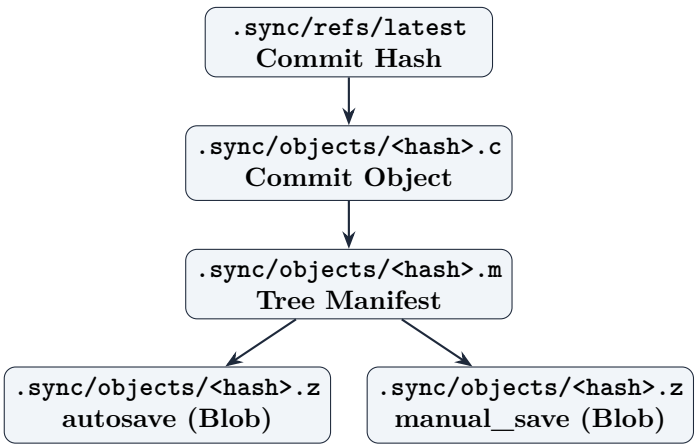


Figure 1: Content-addressable object hierarchy in Shogun-Sync.

The database contains three immutable object types:

- **Blobs (`.z`):** Compressed raw save file contents, named after the SHA-1 hash of the uncompressed data.
- **Trees (`.m`):** Manifests mapping each file in the save folder to its blob hash (`blob <hash> <filename>`).
- **Commits (`.c`):** Commit metadata referencing a tree hash, author hostname, and turn comment (`Tree <hash> Author <host> Comment <msg>`).
- **Refs (`refs/latest`):** Text file containing the 40-character SHA-1 hash of the most recent commit.

2 Cryptographic Subsystem: Custom SHA-1

Shogun-Sync contains a standalone C implementation of the SHA-1 (Secure Hash Algorithm 1) message-digest algorithm (FIPS PUB 180-1) in `kw_sha1.c`. It generates a 160-bit (20-byte, 40-character hex) digest without any external dependencies.

2.1 Padding & Equations (from Design Notes)

The algorithm operates on arbitrary byte streams of length ML bytes ($ML \times 8$ bits). The message must be padded to a multiple of 512 bits (64 bytes), with the last 64 bits reserved for the original message length:

$$\text{diff} = (ML + 1) \pmod{64} \tag{1}$$

$$\text{pad_bytes} = \begin{cases} 56 - \text{diff} & \text{if diff} \leq 56 \\ 64 - \text{diff} + 56 & \text{if diff} > 56 \end{cases} \tag{2}$$

Total buffer size allocated in heap:

$$\text{Size} = ML + 1 + \text{pad_bytes} + 8 \quad \text{bytes} \tag{3}$$

The padding sequence:

1. Byte ML is assigned `0x80` (bit `1` followed by seven `0`s).
2. The subsequent pad_bytes are filled with zeros (`0x00`).
3. The final 8 bytes store the original length in bits ($ML \times 8$) in big-endian byte order:

```
for (size_t i = 0; i < 8; i++) {
    input[ML + diff + 1 + i] = (bits >> (8 * (7 - i))) & 0xFF;
}
```

2.2 Message Expansion & Compression Rounds

Each 512-bit block is parsed into 16 words ($W_0 \dots W_{15}$). The message schedule is expanded to 80 words ($W_{16} \dots W_{79}$) using circular rotation:

$$W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \lll 1 \tag{4}$$

The compression loop uses five 32-bit registers (A, B, C, D, E) initialized with standard IVs:

`H0 = 0x67452301`, `H1 = 0xEFCDAB89`, `H2 = 0x98BADCFE`, `H3 = 0x10325476`, `H4 = 0xC3D2E1F0`

For each round $t \in [0, 79]$:

$$\text{temp} = (A \lll 5) + f_t(B, C, D) + E + K_t + W_t \pmod{2^{32}} \tag{5}$$

$$E = D, \quad D = C, \quad C = B \lll 30, \quad B = A, \quad A = \text{temp} \tag{6}$$

2.3 Compression Step Diagram

Reference: Wikipedia SHA-1 specification (https://en.wikipedia.org/wiki/SHA-1).

3 Compression Subsystem: Zlib

Compression and decompression are implemented in `kw_compress.c` using the Zlib library (version 1.3.2).

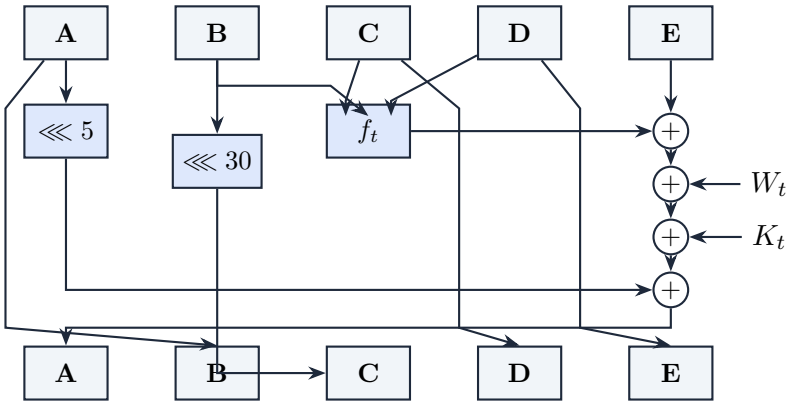


Figure 2: SHA-1 Round Diagram (A, B, C, D, E state update per round t).

3.1 Functions

- `void kw_compress(uint8`$_t * buff, size_t ML, char fileName[])$: $Compresses in-memory raw save data of size$ `ML` $into a non-disk blob file using$ `deflate` $with$ `CHUNK = 16384`
- `bool kw_decompress(uint8`$_t * buff, size_t ML, char fileName[])$: $Decompresses a in-memory buffer (received from network) into a game save file on disk using$ `inflate`.
- `bool kw_decompress_file(char inFileName[], char outFileName[]):` `Reads a compressed file from disk, decompresses it into memory, and writes the output file.`

4 Networking Subsystem: WebSocket & Auto-Resync

Networking is implemented via the Mongoose Library (`mongoose.c`). Connections use TLS 1.3 to connect to the relay server:

`wss://shogun2-sync.duckdns.org/ws/<room>?password=<pass>&action=<action>`

4.1 Auto-Resync Frame Format

When desynchronization occurs, the client requests the save via text frame:

`REQ_SAVE|autosave.save_multiplayer`

The host responds with a binary WebSocket frame (`WEBSOCKET_OP_BINARY`) containing:

Bytes 0–63 (64 bytes)	**Bytes 64–N ($N-64$ bytes)**
Save Filename (ASCII, null-padded)	Zlib-compressed Save Data

The client extracts the filename, passes the remaining bytes to `kw_decompress()`, and restores the save in the game folder.

5 Complete Function and Data Structure Specifications

5.1 Data Structures

- `mapTree` (`kw_conf.h`):

```c
typedef struct {
    char* type; // "blob"
    char* hash; // 40-character hex string
    char* name; // Save filename (e.g. autosave.save_multiplayer)
} mapTree;
```

- package (kw_sha1.h):

```c
typedef struct {
    uint32_t word[80]; // 80 words of expanded message schedule
} package;
```

5.2 Module Inventory

5.2.1 main.c

- `int main(int argc, char *argv[])`: Primary CLI entry point.
- `void generateRoomCode(char *code, size`$_t$*len*) : *Generatesarandom*6−*characterroomcodefrom*ABCD
- `BOOL WINAPI CtrlHandler(DWORD fdwCtrlType)`: Graceful CTRL+C exit handler.

5.2.2 kw_conf.c

- `bool getVariables()`: Retrieves `%APPDATA%` and `%LOCALAPPDATA%`.
- `bool init()`: Creates hidden `.sync` repository directories.
- `bool scan(mapTree **mt, int *x)`: Scans save games directory using Win32 API.
- `bool add(char hashTree[41])`: Compresses saves into blobs and creates tree manifest.
- `bool commit(char comment[])`: Creates commit object and updates `.sync/refs/latest`.
- `bool watch()`: Watches save folder for changes using `FindFirstChangeNotificationA` and polls network.
- `void test(char saveName[])`: Diagnostic routine for save compression.

5.2.3 kw_conn.c

- `void connectToRoom(char roomName[], char password[], char action[])`: Connects to lobby.
- `void pollNetwork()`: Polls Mongoose event loop (`mg_mgr_poll`).
- `void sendPackage(char msg[], size`$_t$*leng*) : *SendstextWebSocketmessage*.
- `void sendSaveToClient(const char *saveName)`: Compresses save and sends binary frame.
- `void eventManager(struct mg_connection *conn, int ev, void *evData)`: Mongoose callback dispatcher.

5.2.4 `kw_sha1.c`

- `void sha1(const uint8_t` $*message, size_t ML, uint32_t result[5])$: $SHA-1 hashing implementation.$
- `void printS(uint32_t` $result[])$: $Prints hash registers in hex.$
- `void resultToChar(uint32_t` $result[], char * resultChar, size_t ML)$: $Formats hash as 40-character hex string.$

6 Third-Party Technologies & Licenses

6.1 Zlib General Purpose Compression Library

Zlib License (Version 1.3.2)

Copyright © 1995–2026 Jean-loup Gailly and Mark Adler
This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.
Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

6.2 Mongoose Embedded Networking Library

Mongoose Library Licensing (Cesanta Software)

Mongoose is dual-licensed under the **GNU General Public License version 2 (GPLv2)** and a Commercial License.

- **Non-Commercial / Open Source:** Fully permitted under GPLv2 for non-commercial and open-source projects.
- **Source Availability:** In accordance with GPLv2, Shogun-Sync releases all integrated source code publicly.
- **Commercial Use:** Closed-source commercial products require a separate Commercial License from Cesanta.